

BÀI 03. MENU VÀ INTENT

Trong bài học trước chúng ta đã tìm hiểu về:

ListView

AlertDialog:

- ✓ DatePickerDialog/TimePickerDialog
- ✓ AlertDialog.Builder
- ✓ Toast

Thực hành thông qua bài kiểm tra thường xuyên 1.

Một số các vấn đề cần quan tâm nhiều hơn:

Tính trải nghiệm người dung: lập trình nút điều khiển, Kiểm tra dữ liệu...

Giao diện người dùng: Bố cục, hình ảnh, màu sắc

Máy tính: Cấu hình, pin, phần mềm...

NỘI DUNG

Menu

- `OptionsMenu`
- `ContextMenu`

Intent

- Explicit intent (`Primary data`, `Object`, `Bundle`, `ActivityResultLaunch...`)
- Implicit intent (`ACTION_VIEW`, `ACTION_SENDTO`, `ACTION_CALL`,...)

3.1. 1. TỔNG QUAN VỀ MENU TRONG ANDROID

Menu là một thành phần giao diện người dùng (UI) cốt lõi, giúp người dùng truy cập các hành động và cài đặt quan trọng trong ứng dụng.

Trong Android, Menu được phân thành ba loại chính, mỗi loại có mục đích và vị trí hiển thị riêng biệt.

1. Options Menu (Menu Tùy chọn)
2. Context Menu (Menu Ngữ cảnh)
3. Popup Menu (Menu Nổi)

1, OPTIONS MENU (MENU TÙY CHỌN)

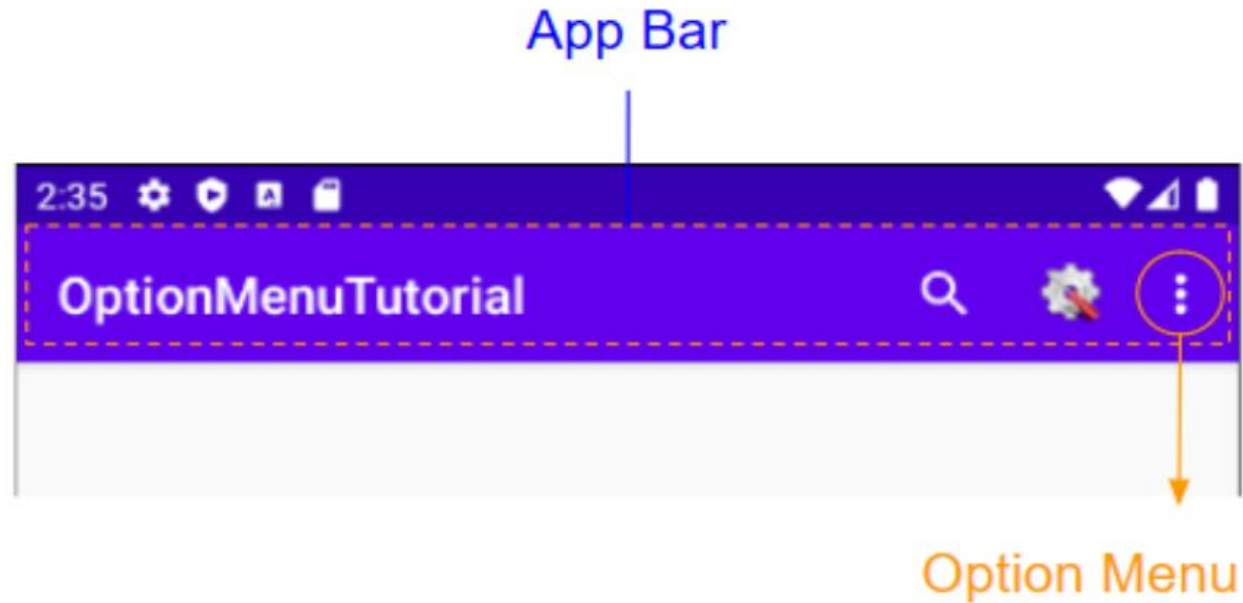
Đây là loại Menu cơ bản và quan trọng nhất, xuất hiện dưới dạng Action Bar (Thanh Hành động) hoặc Toolbar.

Đặc điểm	Chi tiết
Vị trí	Thường nằm ở góc trên cùng bên phải của Activity.
Mục đích	Chứa các hành động toàn cục (Global Actions) của ứng dụng, như Tìm kiếm, Cài đặt (Settings), Chia sẻ, hoặc các chức năng chính khác.
Hiển thị	Các mục quan trọng nhất có thể hiển thị trực tiếp dưới dạng Action Items (biểu tượng) trên Toolbar. Các mục ít quan trọng hơn sẽ được nhóm lại trong Overflow Menu (biểu tượng ba chấm dọc).
Cách tạo	Định nghĩa trong file XML (thường là res/menu/main_menu.xml) và được "phồng" (inflate) vào Activity/Fragment.

1, OPTIONS MENU (MENU TÙY CHỌN)

Cách triển khai chính:

- Ghi đè phương thức `onOptionsItemSelected()` để inflate Menu XML.
- Ghi đè phương thức `onOptionsItemSelected()` để xử lý sự kiện khi người dùng nhấn vào một mục.



2, CONTEXT MENU (MENU NGŨ CẢNH)

Context Menu là menu nổi (floating menu) xuất hiện khi người dùng nhấn giữ (Long Press mặc định là 500 mili giây vào một View cụ thể trong giao diện.

Đặc điểm	Chi tiết
Vị trí	Nổi lên (popup) ngay tại vị trí của View được nhấn giữ.
Mục đích	Cung cấp các hành động chỉ áp dụng cho View đó, ví dụ: Sửa (Edit), Xóa (Delete), Sao chép (Copy) khi nhấn giữ vào một item trong RecyclerView hoặc ListView.
Cách tạo	Đăng ký View đó cho Context Menu bằng <code>registerForContextMenu()</code> .

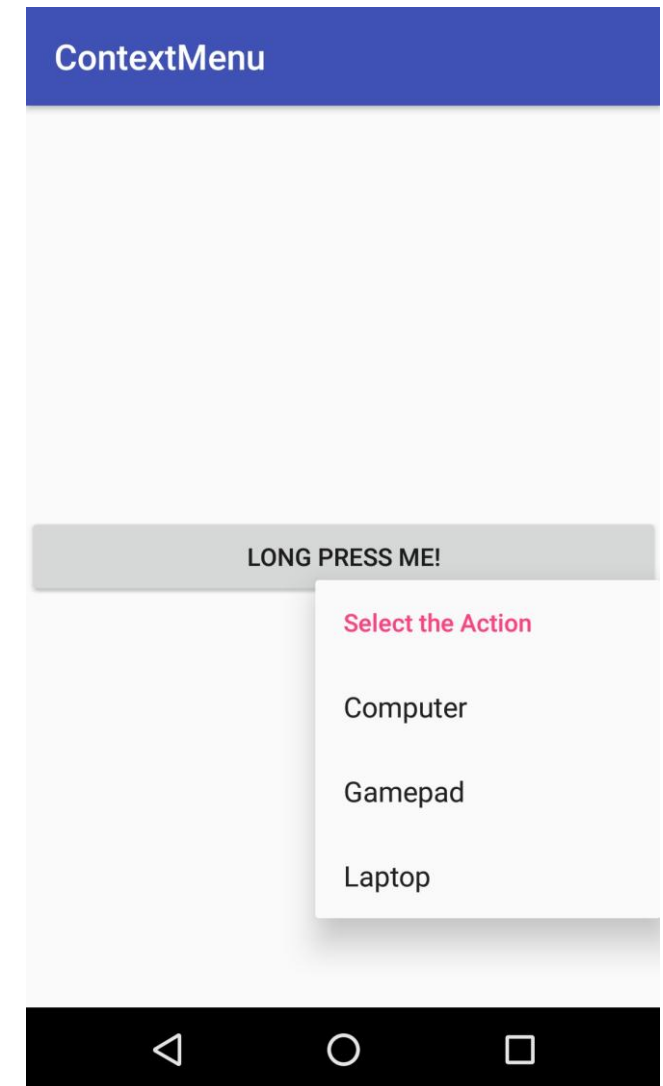
2, CONTEXT MENU (MENU NGŨ CẢNH)

Context Menu có thể chứa nhiều Menu Item và Sub Menu.

Khi một View được đăng ký với sự kiện Long-Press, hệ thống sẽ gọi phương thức **onCreateContextMenu()** để tạo và hiển thị Context Menu.

Cách triển khai chính:

- Ghi đè phương thức `onCreateContextMenu()` để inflate Menu XML hoặc định nghĩa các mục.
- Ghi đè phương thức `onContextItemSelected()` để xử lý sự kiện.



3.1.2. CÁC THUẬT NGỮ

- **Menu:** Interface cơ bản đại diện cho một danh sách các mục Menu (MenuItem).
- **MenuItem:** Interface đại diện cho một mục riêng lẻ trong Menu. Nó chứa các thuộc tính như ID, tiêu đề, và biểu tượng (icon).
- **MenuInflater:** Lớp dùng để đọc (parse) file tài nguyên Menu XML (res/menu/) và chuyển đổi chúng thành đối tượng Menu trong code.
- **Menu XML:** File định nghĩa cấu trúc Menu. Các thẻ chính bao gồm:
 - ✓ <menu>: Phần tử gốc.
 - ✓ <item>: Đại diện cho một MenuItem.
 - ✓ <group>: Dùng để nhóm các mục Menu có chung thuộc tính (như trạng thái checkable).

3.1.2. CÁC KHÁI NIỆM

Lưu ý về Design

Trong thiết kế Android hiện đại, trọng tâm nằm ở Toolbar (thay thế cho Action Bar).

android:showAsAction: Đây là thuộc tính quan trọng nhất trong Menu XML để điều khiển Option Menu. Các giá trị thường dùng là:

- **ifRoom:** Hiển thị dưới dạng biểu tượng nếu có đủ không gian trên Toolbar.
- **always:** Luôn luôn hiển thị dưới dạng biểu tượng.
- **never:** Luôn nằm trong Overflow Menu (ba chấm).

3.1.3. DEMO OPTION MENU

- Sử dụng **ActionBar**: Vào thư mục res/values/themes để cấu hình lại ActionBar (Bỏ NoActionBar)
- Sử dụng **ToolBar**
 - // 1. Tìm Toolbar bằng ID
 - Toolbar toolbar = findViewById(R.id.my_toolbar);
 - // 2. Cài đặt Toolbar làm ActionBar cho Activity
 - setSupportActionBar(toolbar);
 - // (Tùy chọn) Đặt tiêu đề (title) cho Toolbar
 - if (getSupportActionBar() != null) {
 - getSupportActionBar().setTitle("Ứng Dụng Của Tôi");
 - }
- Thiết kế Option menu
- Gọi Option menu trong MainActivity.java
- Bắt sự kiện cho các item menu

18:15 A

A250710_StudentMa... + ⚙

Student Name

☒ Male ☐ Female

Date of birth 🕒

Hobbies

☐ Coding ☐ Reading ☐ Singing

New Edit Remove

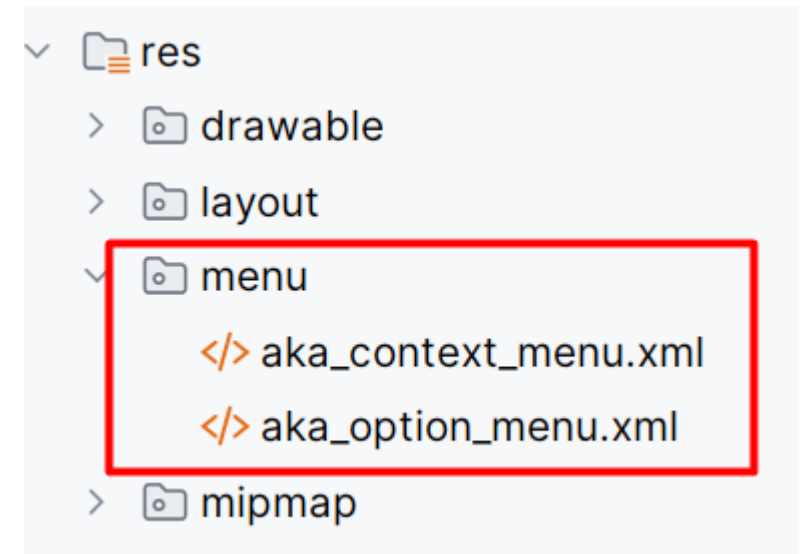
Student{name='aa', sex=Male, dateBirth='01/01/2020', hobby='Coddling'}

3.1.3. OPTION MENU

Ví dụ về cách sử dụng OptionMenu trong Android để hiển thị các tùy chọn trên thanh công cụ (App Bar):

Bước 1. Tạo file menu XML: Trong thư mục res/menu, tạo file main_menu.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
  <item
    android:id="@+id/menu_add"
    android:title="@string/menu_add"
    android:icon="@drawable/add_50dp"
    app:showAsAction="ifRoom" />
  <item
    android:id="@+id/menu_settings"
    android:title="@string/menu_settings"
    android:icon="@drawable/settings_50dp"
    app:showAsAction="ifRoom"
  />
</menu>
```



3.1.3. OPTION MENU

Bước 2: Gọi menu trong MainActivity.java

//Gọi OptionMenu

@Override

```
public boolean onCreateOptionsMenu(Menu menu) {  
    getMenuInflater().inflate(R.menu.aka_option_menu, menu);  
    return super.onCreateOptionsMenu(menu);  
}
```

@Override

```
public boolean onOptionsItemSelected(@NonNull MenuItem item) {  
    if (item.getItemId() == R.id.menu_add) {  
        Toast.makeText(this, "Ban da chon them", Toast.LENGTH_SHORT).show();  
    }  
    if (item.getItemId() == R.id.menu_settings) {  
        Toast.makeText(this, "Ban da chon nut cai dat", Toast.LENGTH_SHORT).show();  
    }  
    return super.onOptionsItemSelected(item);  
}
```

3.1.4. CONTEXT MENU

1. Tạo file menu XML: Tạo file aka_context_menu.xml trong thư mục res/menu:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <item
    android:id="@+id/menu_remove"
    android:title="@string/btn_remove"/>
    <item
      android:id="@+id/menu_share"
      android:title="@string/menu_share">
        <menu>
          <item
            android:id="@+id/menu_facebook"
            android:title="facebook"/>
          <item
            android:id="@+id/menu_zalo"
            android:title="zalo"/>
        </menu>
      </item>
    </menu>
  </menu>
```

3.1.4. CONTEXT MENU

2. Xử lý trong MainActivity.java

```
protected void onCreate(Bundle savedInstanceState) {  
    ...  
    mapping();  
    //Dang ky ContextMenu cho ListView  
    registerForContextMenu(lvStudents);  
    ...}
```

3.1.4. CONTEXT MENU

2. Xử lý trong MainActivity.java

//Sau khi dang ky ConTextMenu cho ListView, viet cac phuong thuc onCreateContextMenu va OnContextItemSelected
@Override

```
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {  
    super.onCreateContextMenu(menu, v, menuInfo);  
    getMenuInflater().inflate(R.menu.aka_context_menu, menu);  
    menu.setHeaderTitle("Select action");  
}
```

@Override

```
public boolean onContextItemSelected(@NonNull MenuItem item) {  
    if(item.getItemId()==R.id.menu_remove){  
        Toast.makeText(this, "Xoa ban ghi nay", Toast.LENGTH_SHORT).show();  
    }  
    if(item.getItemId()==R.id.menu_facebook){  
        Toast.makeText(this, "Chia se tren Facbook", Toast.LENGTH_SHORT).show();  
    }  
    if(item.getItemId()==R.id.menu_zalo){  
        Toast.makeText(this, "Chia se tren zalo", Toast.LENGTH_SHORT).show();  
    }  
    return super.onContextItemSelected(item);  
}
```

3.1.4. CONTEXT MENU

📌 Ghi chú

registerForContextMenu(view) là bắt buộc để kích hoạt ContextMenu cho View đó. ContextMenu thường dùng cho ListView, Button, hoặc TextView. Từ Android 3.0 trở lên, icon trong ContextMenu không được hỗ trợ.

PHẦN 4. INTENT

4.1. Khái niệm và Vai trò Cơ bản của Intent

Trong Android, Intent đóng vai trò như một thông điệp (messaging object) cho phép yêu cầu một hành động từ một thành phần ứng dụng khác (ví dụ: Activity, Service, hoặc Broadcast Receiver). Intent không chỉ giới hạn trong một ứng dụng, mà còn cho phép các ứng dụng giao tiếp với nhau, tạo nên tính linh hoạt và mở của hệ điều hành Android

Intent là gì?

Intent là một đối tượng chứa thông tin mô tả về một hành động cần thực hiện. Nó là một mong muốn hay ý định của người gửi (một thành phần ứng dụng) được gửi đến hệ thống Android, và hệ thống sẽ cố gắng tìm ra thành phần phù hợp để thực hiện mong muốn đó



PHẦN 4. INTENT

- **Vai trò Chính của Intent**

- ✓ Khởi động một Activity: Đây là chức năng phổ biến nhất. Intent được dùng để chuyển từ màn hình (Activity) này sang màn hình khác, có hoặc không kèm dữ liệu.
- ✓ Khởi động một Service: Intent có thể được dùng để bắt đầu một hoạt động chạy ngầm (Service) hoặc truyền dữ liệu đến một Service đang chạy.
- ✓ Phát đi một Broadcast (Broadcast an Intent): Intent được gửi đến một nhóm các thành phần Broadcast Receiver đã đăng ký để thông báo về một sự kiện xảy ra trong hệ thống hoặc ứng dụng (ví dụ: pin yếu, kết nối mạng thay đổi).

PHẦN 4. INTENT

4.2 Các Loại Intent (Explicit vs. Implicit)

Intent được chia thành hai loại chính, phân biệt dựa trên cách nó xác định thành phần đích:

1. Explicit Intent (Intent Tường minh)

- Một Explicit Intent chỉ rõ tên lớp (Class Name) của thành phần đích mà nó muốn khởi động.
- Phạm vi: Loại Intent này hầu như chỉ được sử dụng trong nội bộ một ứng dụng, nơi bạn biết chính xác Activity hay Service nào bạn muốn gọi.
- Ví dụ: Khởi động DetailActivity từ MainActivity.

```
Intent intent = new Intent(MainActivity.this, DetailActivity.class);  
startActivity(intent);
```

4.2 CÁC LOẠI INTENT (EXPLICIT VS. IMPLICIT)

Intent được chia thành hai loại chính, phân biệt dựa trên cách nó xác định thành phần đích:

1, Explicit Intent (Intent Thường minh)

- Một Explicit Intent chỉ rõ tên lớp (Class Name) của thành phần đích mà nó muốn khởi động.
- Phạm vi: Loại Intent này hầu như chỉ được sử dụng trong nội bộ một ứng dụng, nơi bạn biết chính xác Activity hay Service nào bạn muốn gọi.
- Ví dụ: Khởi động DetailActivity từ MainActivity.

```
Intent intent = new Intent(MainActivity.this, DetailActivity.class);  
startActivity(intent);
```

4.2 CÁC LOẠI INTENT (EXPLICIT VS. IMPLICIT)

2, Implicit Intent (Intent Ngầm định)

- **Implicit Intent** không chỉ định rõ tên lớp của thành phần đích, mà thay vào đó mô tả **hành động (Action)** cần thực hiện và **dữ liệu (Data)** liên quan.
- **Phạm vi:** Hệ thống Android sẽ xem xét tất cả các thành phần ứng dụng (của ứng dụng hiện tại và các ứng dụng khác) đã được khai báo (qua **Intent Filter** trong AndroidManifest.xml) để xem thành phần nào có thể xử lý Intent này.
- **Ví dụ:** Yêu cầu hiển thị một trang web.

```
Intent intent = new Intent(Intent.ACTION_VIEW,  
Uri.parse("https://www.google.com"));  
startActivity(intent);
```

Hệ thống sẽ tìm các ứng dụng có khả năng xử lý ACTION_VIEW cho kiểu dữ liệu http (ví dụ: trình duyệt web).

4.3. CÁC THÀNH PHẦN CẤU TẠO CỦA MỘT INTENT

Một đối tượng Intent chứa một số thông tin quan trọng mà hệ thống Android sử dụng để xác định thành phần đích và cách thức hoạt động.

1, Component Name (Tên Thành phần)

Chỉ được sử dụng trong Explicit Intent. Nó là tên đủ điều kiện của lớp đích (ví dụ: `com.example.myapp.DetailActivity`).

2. Action (Hành động)

Là một chuỗi mô tả hành động chung cần thực hiện. Quan trọng nhất đối với Implicit Intent. Android định nghĩa nhiều hành động tiêu chuẩn:

- `Intent.ACTION_VIEW`: Hiển thị dữ liệu cho người dùng.
- `Intent.ACTION_SEND`: Chia sẻ dữ liệu.
- `Intent.ACTION_DIAL`: Mở giao diện gọi điện.
- `Intent.ACTION_MAIN`: Điểm vào chính của Activity.

4.4. VÍ DỤ SỬ DỤNG INTENT

Explicit Intent – mở Activity khác:

```
Intent intent = new Intent(this, SecondActivity.class);  
intent.putExtra("username", "CongDH");  
startActivity(intent);
```

Implicit Intent – mở trình duyệt:

```
Intent intent = new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("https://www.microsoft.com"));
```

4.4. VÍ DỤ SỬ DỤNG INTENT

4.2.3.1. Truyền dữ liệu qua Intent

- Dùng `putExtra()` để gửi dữ liệu: `intent.putExtra("key", "value");`
- Dùng `getIntent().getStringExtra("key")` để nhận dữ liệu ở Activity đích.

4.2.3.2. Nhận kết quả từ Activity khác

- Gọi `startActivityForResult(intent, REQUEST_CODE)`
- Nhận kết quả trong `onActivityResult()`

4.5. DEMO VỚI EXPLICIT INTENT

1, Dữ liệu nguyên thủy.

activity_main.xml

```
<Button  
    android:id="@+id/btn_go"  
    android:text="Đi đến màn hình kế tiếp"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"/>
```

MainActivity.java

4.5. DEMO VỚI EXPLICIT INTENT

MainActivity.java

```
btnGo.setOnClickListener(new View.OnClickListener() {  
    public void onClick(View view) {  
        sendData();  
    } });  
  
// Chuyen du lieu di  
private void sendData(){  
    intent=new Intent(MainActivity.this,SecondActivity.class);  
    intent.putExtra("msg","Loi chao tu MainActivity");  
    startActivity(intent);  
}
```

4.5. DEMO VỚI EXPLICIT INTENT

activity_second.xml

```
<TextView
    android:id="@+id/tv_msg"
    android:text="Chua co du lieu"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
```

SecondActivity.java

```
mapping();
String msg=getIntent().getStringExtra("msg");
tvMsg.setText(msg);
```

4.5. DEMO VỚI EXPLICIT INTENT

2, Intent với dữ liệu là Object

Lưu ý đối tượng Product phải cài đặt **Serializable**

```
public class Product implements Serializable {  
    private int id;  
    private String name;  
    private
```

MainActivity.java

```
myintent= new Intent(MainActivity.this,ObjIntentSub.class);  
Product obj =new Product();  
obj.setId(Integer.parseInt(edtID.getText().toString()));  
obj.setName(edtName.getText().toString());  
obj.setPrice(Float.parseFloat(edtPrice.getText().toString()));  
myintent.putExtra("productObj",obj);  
startActivity(myintent);
```

SubActivity.java

```
intentObj = getIntent();  
Product obj = new Product();  
obj = (Product)  
intentObj.getSerializableExtra("productObj");  
txtIdObj.setText("" + obj.getId());  
txtNameObj.setText(obj.getName());  
txtPriceObj.setText("" + obj.getPrice());
```

4.5. DEMO VỚI EXPLICIT INTENT

3, Sử dụng bundle để chuyển dữ liệu

MainActivity.java

```
//demo voi intent su dung Bundle
btnBundle.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        bundle=new Bundle();
        bundle.putInt("id",Integer.parseInt(edtID.getText().toString()));
        bundle.putString("name",edtName.getText().toString());

        bundle.putFloat("price",Float.parseFloat(edtPrice.getText().toString(
        )));
        myintent=new Intent(MainActivity.this,
        BundleIntentSubActivity.class);
        myintent.putExtra("myBundle",bundle);
        startActivity(myintent);
    }
});
```

BundleIntentSubActivity.java

```
...
intentBundle=getIntent();
myBundle=intentBundle.getBundleExtra("myBundl
e");
txtIdBundle.setText(""+myBundle.getInt("id"));
txtNameBundle.setText(myBundle.getString("nam
e"));
txtPriceBundle.setText(""+myBundle.getFloat("pri
ce"));

btnOkBundle.setOnClickListener(new
View.OnClickListener() {
    @Override
    public void onClick(View v) {
        finish();
    }
});
```

4.5. DEMO VỚI EXPLICIT INTENT

4, Sử dụng `ActivityResultLauncher` để nhận kết quả
Để nhận kết quả từ `SecondActivity`, có thể sử dụng `ActivityResultLauncher` để thay thế cho `startActivityForResult()` đã bị deprecated.

Ví dụ: Khi nhấn vào menu Settings, hiện `activity_settings` cho phép chọn màu nền cho `activity_main`



4.5. DEMO VỚI EXPLICIT INTENT

Trong MainActivity.java – Đăng ký và sử dụng ActivityResultLauncher

//Khai báo đối tượng ActivityResultLauncher;

```
private ActivityResultLauncher<Intent> launcher;
```

Trong onCreate() method định nghĩa đối tượng launcher;

```
launcher = registerForActivityResult(  
    new ActivityResultContracts.StartActivityForResult(), activityResult -> {  
        if (activityResult.getResultCode() == RESULT_OK && activityResult.getData() != null) {  
            Toast.makeText(this, "Nhan ket qua tra ve", Toast.LENGTH_SHORT).show();  
            int selectedColor = activityResult.getData().getIntExtra("selectedColor", Color.GRAY);  
            linearLayout.setBackgroundColor(selectedColor);  
        }  
    }  
);
```

4.5. DEMO VỚI EXPLICIT INTENT

Trong menu Settings - `onOptionsItemSelected()`, gọi `activity_settings`

```
public boolean onOptionsItemSelected(@NonNull MenuItem item) {  
    if (item.getItemId() == R.id.menu_add) {  
        Toast.makeText(this, "Bạn đã chọn Add", Toast.LENGTH_SHORT).show();  
    }  
    if (item.getItemId() == R.id.menu_settings) {  
        Intent intent2 = new Intent(MainActivity.this, SettingsActivity.class);  
        intent2.putExtra("msgSettings", "Chon background");  
        launcher.launch(intent2);  
    }  
    return super.onOptionsItemSelected(item);  
}
```


4.5. DEMO VỚI EXPLICIT INTENT

Trong SettingActivity.java

Trong onCreate() method của SettingActivity.java, nhận giá trị gửi đến từ MainActivity.java

```
mapping();
```

```
String msgSettings = getIntent().getStringExtra("msgSettings");
```

```
Toast.makeText(this, msgSettings, Toast.LENGTH_SHORT).show();
```

Trong định nghĩa sự kiện của nút btnOk

4.5. DEMO VỚI EXPLICIT INTENT

```
btnOk.setOnClickListener(new View.OnClickListener() {
    public void onClick(View view) {
        int selectedColor = Color.YELLOW;
        if (rgColor.getCheckedRadioButtonId() == R.id.rd_blue) {
            selectedColor = Color.BLUE;
        }
        if (rgColor.getCheckedRadioButtonId() == R.id.rd_green) {
            selectedColor = Color.GREEN;
        }
        if (rgColor.getCheckedRadioButtonId() == R.id.rd_red) {
            selectedColor = Color.RED;
        }
        Intent resultIntent = new Intent();
        resultIntent.putExtra("selectedColor", selectedColor);
        setResult(RESULT_OK, resultIntent);
        finish();//Tra ket qua va ket thuc activity_settings
    }
});
```

CHÚ Ý

- `ActivityResultLauncher` giúp mã ngắn gọn, dễ bảo trì hơn so với `startActivityForResult`.
- Bạn có thể dùng nhiều launcher khác nhau cho các mục đích như chọn ảnh, xin quyền truy cập, v.v.